```
{
    atomic() noexcept = default;
    constexpr atomic(BaseType) noexcept;
    BaseType operator=(BaseType) volatile noexcept;
    BaseType operator=(BaseType) noexcept;

    atomic(const atomic&) = delete;
    atomic& operator=(const atomic&) = delete;
    atomic& operator=(const atomic&) volatile = delete;

    bool is_lock_free() const volatile noexcept;
    bool is_lock_free() const noexcept;
    void store(BaseType,memory_order = memory_order_seq_cst)
        volatile noexcept;
    void store(BaseType,memory_order = memory_order_seq_cst)
noexcept;
    BaseType load(memory_order = memory_order_seq_cst)
        const volatile noexcept;
    BaseType load(memory_order = memory_order_seq_cst) const
noexcept;
    BaseType exchange(BaseType,memory_order = memory_order_seq_cst)
        volatile noexcept;
    BaseType exchange(BaseType,memory_order = memory_order_seq_cst)
        noexcept;

    bool compare_exchange_strong(
        BaseType & old_value, BaseType new_value,
        memory_order order = memory_order_seq_cst) volatile
noexcept;
    bool compare_exchange_strong(
        BaseType & old_value, BaseType new_value,
        memory_order order = memory_order_seq_cst) noexcept;
    bool compare_exchange_strong(
        BaseType & old_value, BaseType new_value,
        memory_order success_order,
        memory_order failure_order) volatile noexcept;
    bool compare_exchange_strong(
        BaseType & old_value, BaseType new_value,
        memory_order success_order,
        memory_order failure_order) noexcept;
    bool compare_exchange_weak(
        BaseType & old_value, BaseType new_value,
        memory_order order = memory_order_seq_cst)
        volatile noexcept;
    bool compare_exchange_weak(
        BaseType & old_value, BaseType new_value,
        memory_order order = memory_order_seq_cst) noexcept;
    bool compare_exchange_weak(
        BaseType & old_value, BaseType new_value,
        memory_order success_order,
```